

Datavalidering med Pandera

Production Data Quality Inspector | Yunus Emre Capar | September 2026

Kort sammanfattning

Jag har byggt ett Pythonflöde som granskar produktionsdata innan informationen används i rapporter eller beslut. Pandera är huvudverktyget. Streamlit visar resultatet och en manuell Pandas-version används för jämförelse.

Syfte

Mitt syfte var att lära mig hur Pandera kan användas som ett tydligt datakontrakt för en Pandas DataFrame. Jag ville undersöka hur man kan kontrollera kolumner, datatyper och affärsregler på ett återanvändbart sätt. Jag ville också förstå skillnaden mellan Pandera och kontroller som skrivs manuellt med Pandas.

Arbetet är medvetet avgränsat. Lösningen läser en CSV-fil med produktionsdata, kör 19 valideringsregler och skapar ett resultat som användaren kan undersöka. Jag har inte byggt databas, inloggning, automatisk rättning eller ett komplett produktionssystem.

Området och dess relevans

Data Science börjar ofta med data som kommer från andra system. Om datan innehåller tomma ID-nummer, fel datum, okända maskiner eller orimliga mängder kan senare analys bli missvisande. En snygg rapport hjälper inte om indata är fel. Därför behöver datakvalitet kontrolleras tidigt.

Detta är särskilt relevant i produktion. Ett fel i kolumnen producerad mängd kan påverka OEE, kapacitetsplanering och uppföljning av kassation. Datavalidering hjälper en dataanalytiker eller Data Scientist att upptäcka problemet innan informationen används vidare.

Viktiga begrepp

- Schema: ett kontrakt som beskriver vilka kolumner och regler datan ska följa.
- Valideringsregel: en kontroll som ger godkänt eller ett tydligt fel, till exempel att cykeltid måste vara större än noll.
- DataFrame: Pandas tabellformat med rader och kolumner.
- Lazy validation: Pandera fortsätter efter första felet och samlar flera oberoende problem.
- Cross-column check: en regel som jämför flera kolumner, till exempel `good + scrap <= produced`.

Genomförande

Jag skapade först en syntetisk CSV-fil med 1 200 produktionsrader och 13 kolumner. Informationen liknar ett enkelt utdrag från en fabrik: datum, produktionslinje, maskin, skift, produktkod, producerad mängd, godkänd mängd, kassation, stopptid och cykeltid. Datan innehåller inga personuppgifter eller verkliga företagsvärden.

Jag planterade ett litet antal fel för att kunna visa hur valideringen fungerar. Exempel är ett ogiltigt datum, ett saknat record_id, ett dubbelt ID, maskinen MC-999, skiftet Weekend, negativa värden, texten unknown i en mängdkolumn och texten fast i cykeltid.

Projektets delar

Del	Ansvar
config.py	Gemensam konfiguration för kolumner och tillåtna värden.
schema.py	Bygger Pandera-schemat och kopplar kontroller till regler.
validation.py	Kör Pandera och översätter resultatet till tydliga problem.
manual_validation.py	Gör samma kontroll med vanlig Pandas-kod för jämförelse.
quality.py	Räknar rena rader, påverkade rader och kvalitetspoäng.
reporting.py	Skapar tabeller och CSV-rapporter utan att skriva över källdatan.
streamlit_app.py	Visar arbetsflödet i webbläsaren.

Viktiga tekniska val

Pandera är huvudmotor eftersom ett schema samlar kontraktet på ett tydligt ställe. Streamlit är bara gränssnittet. Det betyder att samma validering kan användas från tester eller annan Pythonkod utan att starta webbappen.

Dashboarden använder lazy=True. Direkt validering stoppar efter första allvarliga felet, men i en filkontroll vill användaren oftast se flera problem på samma gång. Därför passar lazy validation bättre här.

Jag valde att inte rätta fel automatiskt. Om texten unknown blir noll utan att användaren märker det kan rapporten se bättre ut men innehålla fel information. Lösningen visar därför originalvärdet och raden där problemet finns.

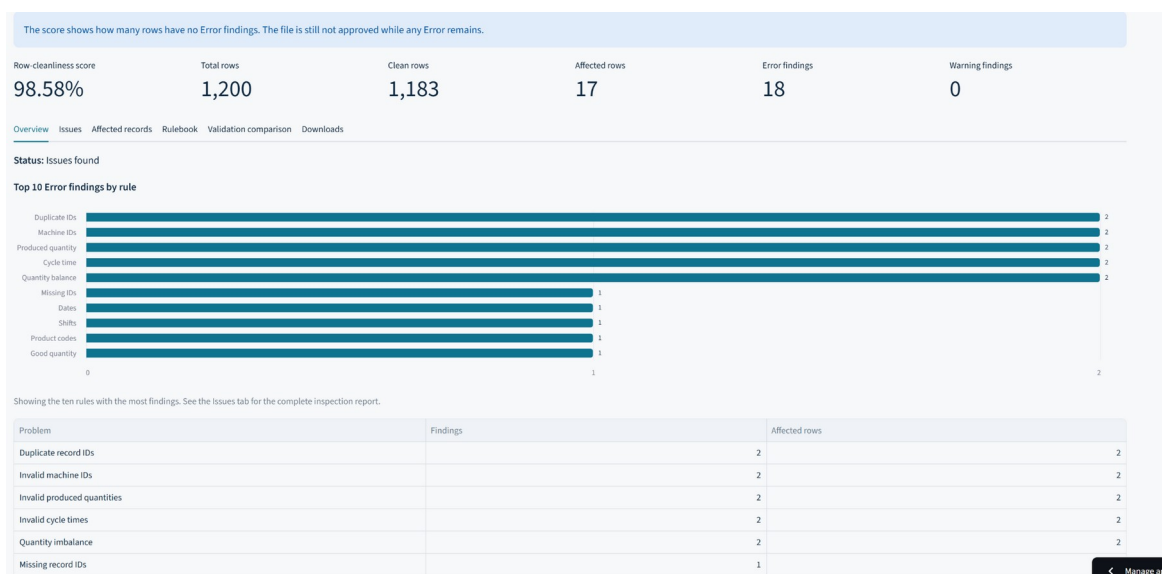
Oväntade extra kolumner behandlas som Warning. Filen kan fortfarande innehålla alla 13 nödvändiga kolumner även om en extra kolumn finns. De andra affärsproblemen behandlas som Error.

Resultat

Lösningen går att köra lokalt och är publicerad som en Streamlit-app. Användaren kan välja demonstrationsfilen eller ladda upp en egen CSV-fil. Efter kontrollen visas status, kvalitetspoäng, problem per regel och de originalrader som påverkas. Fyra CSV-rapporter kan laddas ner direkt från minnet.

Mätvärde	Resultat
Status	Issues found
Totalt antal rader	1 200
Rena rader	1 183
Påverkade rader	17
Error findings	18
Warning findings	0
Row-cleanliness score	98,58 %

Det finns 18 fel men bara 17 påverkade rader eftersom en rad kan bryta mot flera regler. Poängen räknar varje påverkad rad en gång. Filen har därför 1 183 rena rader av 1 200. Trots 98,58 procent är status fortfarande Issues found eftersom minst ett Error finns kvar.



Figur 1. Översikten visar poäng, antal problem och de tio vanligaste felreglerna.

Jämförelse

Direkt Pandera hittade det första felet och stoppade. Komplet Pandera hittade 18 fel på 17 rader inom 13 regelkategorier. Den manuella Pandas-versionen hittade samma affärsresultat, men den behövde mer egen kod för varje kontroll och för felrapporteringen.

Testresultat

159 automatiserade tester gick igenom i den slutliga kontrollen. Testerna kontrollerar bland annat schema, laddning, båda validerarna, rapporter, poäng och Streamlit-flödet.

Begränsningar och möjliga förbättringar

Lösningen visar ett tydligt arbetssätt, men den är inte ett komplett system för en fabrik. Reglerna är anpassade till demonstrationsdatan. Ett annat företag kan ha andra maskiner, skift, produktkoder och gränsvärden. Då måste konfigurationen ändras och reglerna godkännas av verksamheten.

Ett godkänt schema bevisar inte att alla värden är sanna. En rad kan till exempel ha maskinen MC-101 och ett tillåtet antal, men ändå beskriva fel verklig händelse. Validering kontrollerar det vi kan formulera, inte allt som kan vara fel.

Kvalitetspoängen är enkel att förstå, men den visar inte hur dyrt eller allvarligt varje fel är. Ett fel i en kritisk batch kan vara viktigare än flera mindre fel. En framtida version skulle kunna använda viktning, men då måste verksamheten bestämma vikterna. Annars kan poängen ge falsk trygghet.

Appen sparar ingen historik och har ingen inloggning. Uppladdade filer hanteras i minnet. Det passar en demonstration och minskar risken att data blir kvar, men det räcker inte för en företagslösning med spårbarhet och flera användare.

Möjliga nästa steg

- Flytta tillåtna värden och gränser till en versionsstyrd konfigurationsfil.
- Lägga till historik så att datakvalitet kan följas över tid.
- Koppla valideringen till ett automatiskt dataflöde före rapportering.
- Lägga till inloggning och behörigheter om verklig produktionsdata används.
- Skapa en separat och godkänd rättningsprocess i stället för att ändra data automatiskt.

Koppling till yrkesrollen

En Data Scientist eller dataanalytiker behöver kunna bedöma om data går att använda. Datavalidering kan läggas före analys, dashboard, maskininlärning eller rapportgenerering. Då upptäcks problem när de kommer in i flödet i stället för efter att någon redan har tagit ett beslut.

I en produktionsmiljö kan samma arbetssätt användas för skiftdata, stopporsaker, kassation, kapacitet och kvalitetsresultat. Reglerna fungerar som en gemensam överenskommelse mellan verksamheten och datateamet. Verksamheten beskriver vad som är rimligt och Python gör kontrollen på samma sätt varje gång.

Min erfarenhet från produktion hjälpte mig att välja regler som går att förklara för en användare. Exempelvis är $\text{good} + \text{scrap} \leq \text{produced}$ lätt att koppla till ett verkligt flöde. Det gör också tekniken lättare att diskutera med personer som inte arbetar med programmering.

Självreflektion

- 1. Vad lärde du dig som du inte kunde innan?** Jag lärde mig hur ett Pandera-schema kan fungera som ett datakontrakt. Jag förstod också bättre hur lazy validation samlar flera fel och hur resultatet kan översättas till information som en användare förstår.
- 2. Vad var svårast att förstå eller genomföra?** Det svåraste var blandade datatyper och regler som använder flera kolumner. Om ett fält innehåller text i stället för ett tal måste lösningen rapportera typfelet utan att samma värde skapar många otydliga följdfel.
- 3. Vilket tekniskt val är du mest nöjd med och varför?** Jag är mest nöjd med att Pandera är huvudmotor och att Streamlit bara visar resultatet. Det gör koden lättare att testa och återanvända. Jag är också nöjd med att originalvärden aldrig rättas i bakgrunden.
- 4. Vad hade du gjort annorlunda om du började om?** Jag hade bestämt resultatmodellen och rapportformatet tidigare. Då hade det varit enklare att bygga Pandera-versionen, Pandas-jämförelsen och dashboarden runt samma struktur från början.
- 5. Vad skulle vara ett naturligt nästa steg?** Nästa steg skulle vara versionsstyrd regelkonfiguration och automatisk körning när en ny fil kommer in. Jag skulle också vilja spara historik så att man kan se om datakvaliteten förbättras.
- 6. Vilket betyg tycker du att arbetet motsvarar?** Jag bedömer att arbetet motsvarar VG.
- 7. Hur kopplar du bedömningen till kraven för G och VG?** Projektet uppfyller G genom ett relevant och avgränsat område, en fungerande lösning, dokumentation, data och tydliga körinstruktioner. Jag motiverar VG med att jag inte bara visar Pandera. Jag förklarar varför schema och lazy validation används, jämför med en alternativ Pandas-lösning, diskuterar tekniska konsekvenser och visar begränsningar. Koden är uppdelad i tydliga moduler och skyddas av tester. Den muntliga presentationen behöver också visa att jag själv förstår dessa val.

Min viktigaste slutsats

Datavalidering handlar inte bara om att hitta fel. Ett bra valideringsflöde ska också förklara var felet finns, vilken regel som bröts och varför problemet är viktigt.

Avslutning

Arbetet gav mig en tydligare bild av hur Python kan användas för att skapa kontroll före analys. Pandera gjorde reglerna samlade och tydliga. Jämförelsen med Pandas visade att samma resultat går att bygga manuellt, men att ett schema ger bättre struktur när kontrollerna blir fler. Lösningen är liten, men den går att köra, testa och demonstrera.

Källor

Pandera documentation - DataFrame Schemas

https://pandera.readthedocs.io/en/stable/dataframe_schemas.html

Använd för schema, kolumner, null-värden, datatyper och checks.

Pandera documentation - Lazy Validation

https://pandera.readthedocs.io/en/stable/lazy_validation.html

Använd för skillnaden mellan direkt och komplett validering.

Pandas documentation - Working with missing data

https://pandas.pydata.org/docs/user_guide/missing_data.html

Använd för hantering och tolkning av saknade värden.

Streamlit documentation - File uploader

https://docs.streamlit.io/develop/api-reference/widgets/st.file_uploader

Använd för CSV-uppladdningen i gränssnittet.

Streamlit documentation - Download button

https://docs.streamlit.io/develop/api-reference/widgets/st.download_button

Använd för rapporter som skapas och laddas ner från minnet.

pytest documentation - Get Started

<https://docs.pytest.org/en/stable/getting-started.html>

Använd för teststruktur och kontroll av förväntade resultat.

Webbkällorna kontrollerades den 11 september 2026.

Projektlänkar

GitHub: <https://github.com/JunusEmre/production-data-quality-inspector>

Live demo: <https://appuction-data-quality-inspector.streamlit.app/>